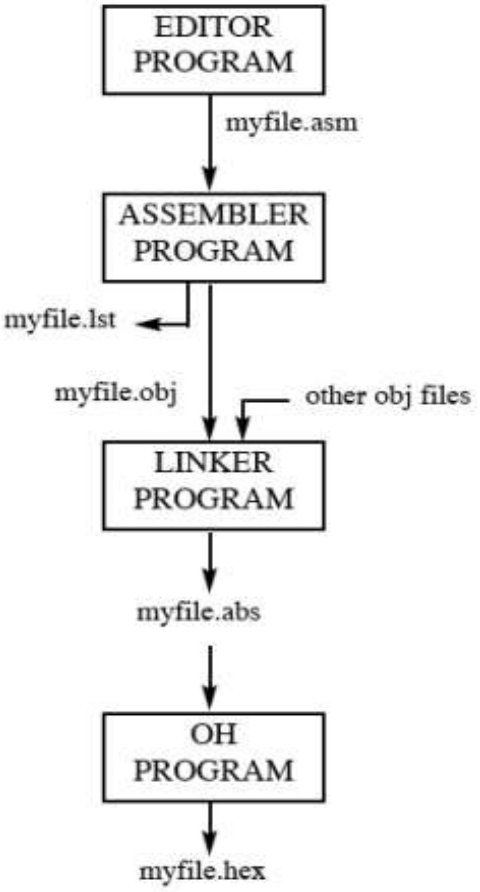
Microcontrollers

ALP

Steps in Assembly Language Programming

1. Use an editor to type in a program "myfile.asm" (may use other extensions)
2. The assembly source program is fed to an 8051 assembler. "myfile.lst" and "myfile.obj" are generated by the assembler.
3. A link program takes one or more object files to produce an absolute object file "myfile.abs". These abs files are used by 8051 trainers that have a monitor program.
4. The "abs"file is fed into a program called "OH" (object to hex converter) which creates a file "myfile.hex"
5. The "myfile.hex" file is to be burned into ROM by a special burner.

- New Windows-based assemblers combine 2-4 into one step



Addition

```
0000         ORG   0H          ;start at location 0
0000 7D25    MOV   R5,#25H     ;load 25H into R5
0002 7F34    MOV   R7,#34H     ;load 34H into R7
0004 7400    MOV   A,#0        ;load 0 into A
0006 2D      ADD   A,R5        ;add contents of R5 to A
                               ;now A = A + R5
0007 2F      ADD   A,R7        ;add contents of R7 to A
                               ;now A = A + R7
0008 2412    ADD   A,#12H      ;add to A value 12H
                               ;now A = A + 12H
000A 80FE HERE:SJMP HERE          ;stay in this loop
000A         END               ;end of asm source file
```

Address	Code
0000	7D
0001	25
0002	7F
0003	34
0004	74
0005	00
0006	2D
0007	2F
0008	24
0009	12
000A	80
000B	FE

Code Memory

ROM Contents

Address	Code
0000	7D
0001	25
0002	7F
0003	34
0004	74
0005	00
0006	2D
0007	2F
0008	24
0009	12
000A	80
000B	FE

PSW –Flag Register

PSW

CY	AC	F0	RS1	RS0	OV	--	P

CY	PSW.7	Carry flag.
AC	PSW.6	Auxiliary carry flag.
F0	PSW.5	Available to the user for general purpose.
RS1	PSW.4	Register Bank selector bit 1.
RS0	PSW.3	Register Bank selector bit 0.
OV	PSW.2	Overflow flag.
--	PSW.1	User-definable bit.
P	PSW.0	Parity flag. Set/cleared by hardware each instuction cycle to indicate an odd/even number of 1 bits in the accumulator.

RS1	RS0	Register Bank	Address
0	0	0	00H - 07H
0	1	1	08H - 0FH
1	0	2	10H - 17H
1	1	3	18H - 1FH

PSW

PSW BIT FORMAT

MSB LSB

D7	D6	D5	D4	D3	D2	D1	D0
CY	AC	F0	RS1	RS0	OV	–	P

BIT DESCRIPTIONS

Bit	Name	Description
D7	CY (Carry Flag)	Set to 1 if there is a carry out from the MSB during addition or a borrow during subtraction. Otherwise 0.
D6	AC (Auxiliary Carry)	Set if there is a carry from bit 3 to bit 4. Used mainly for BCD arithmetic (DA A instruction).
D5	F0 (User Flag)	General purpose flag for programmer use.
D4	RS1 (Register Select 1)	Used with RS0 to select one of the four register banks.
D3	RS0 (Register Select 0)	Used with RS1 to select one of the four register banks.
D2	OV (Overflow Flag)	Set if signed arithmetic causes overflow.
D1	– (Reserved)	Reserved for future use. Normally cleared to 0.
D0	P (Parity Flag)	Automatically updated after every instruction. Set to 1 if accumulator has odd number of 1s, else 0.

Register Bank Selection

REGISTER BANK SELECTION (Using RS1, RS0)

RS1	RS0	Register Bank	RAM Address (R0–R7)
0	0	Bank 0	00H – 07H
0	1	Bank 1	08H – 0FH
1	0	Bank 2	10H – 17H
1	1	Bank 3	18H – 1FH

Example: To select Register Bank 1

```
SETB   PSW.3     ; RS0 = 1
CLR    PSW.4     ; RS1 = 0
```

8 Bit Addition-WAP to add two immediate data and store in external memory

ALGORITHM:

1. Clear C – register for Carry
2. Get the data immediately .
3. Add the two data
4. Store the result in memory pointed by DPTR

PROGRAM:

```
        ORG     4100
        CLR     C
        MOV     A,#data1
        ADD     A,#data2
        MOV     DPTR,#4500
        MOVX    @DPTR,A
HERE:   SJMP    HERE
```

Multiply contents of Reg A&B and store the result at 4500 and 4501.

Address	Label	Mnemonics	Comments
4000		MOV A, #0A	Move 0A to accumulator
4002		MOV B, #88	Move 88 to A
4005		MUL AB	Multiply the contents of A with B
4006		MOV DPTR, #4500	Point data pointer to memory location
4009		MOVX @DPTR, A	Move accumulator contents to memory pointed by data pointer
400A		INC DPTR	Increment data pointer
400B		MOV A, B	Move the contents of B to A
400D		MOVX @DPTR, A	Move the contents of A to memory pointed by data pointer
400E	HLT:	SJMP HLT	Short jump here

Input values	Output	Value
0A	4500	50
88	4501	05

To divide two bytes stored in registers A and B, and store the results in 4500 and 4501

Address	Label	Mnemonics	Comments
4000		MOV A, #65	Move 65 to accumulator
4002		MOV B, #08	Move 08 to B
4005		DIV AB	Divide the contents of A with B
4006		MOV DPTR, #4500	Point data pointer to memory location
4009		MOVX @DPTR, A	Move accumulator contents to memory pointed by data pointer
400A		INC DPTR	Increment data pointer
400B		MOV A, B	Move the contents of B to A
400D		MOVX @DPTR, A	Move the contents of A to memory pointed by data pointer
400E	HLT:	SJMP HLT	Short jump here

Input values	Output	Value
0A	4500	0C
88	4501	05

Write an assembly language program to exchange N = ___h bytes of data at location A: _____h and at location B: _____h.

```
//Let N = 05h A: 30h B: 40h
        mov r0,#30h //source address
        mov r1,#40h //destination address
        mov r7,#05h //count, the number of data to be exchanged
back:   mov a,@r0
        mov r4,a
        mov a,@r1
        mov @r0,a
        mov a,r4
        mov @r1,a
        inc r0
        inc r1
        djnz r7,back
        end
```

Transfer the block of data from 20h to 30h to external location 1020h to 1030h.

```
        Mov r7, #0Ah          ; initialize counter by 10d
            Mov r0, #20h          ; get initial source location
            Mov dptr, #1020h      ; get initial destination location
Nxt:    Mov a, @r0            ; get first content in acc
            Movx @dptr, a         ; move it to external location
            Inc r0                ; increment source location
            Inc dptr              ; increase destination location
              Djnz r7, nxt      ; decrease r7. if zero then over otherwise move next
```